



Neural Networks and Machine Learning

Perceptrons

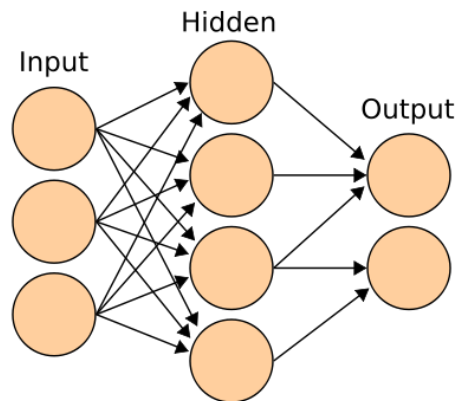
What is a perceptron?

Confusion in the literature about what exactly constitutes a perceptron.

The term was first introduced by Rosenblatt (1962) to describe a specific type of feed-forward neural net with a grid of binary inputs and a threshold binary output unit.

Now often used as a general term for layered feed-forward networks.

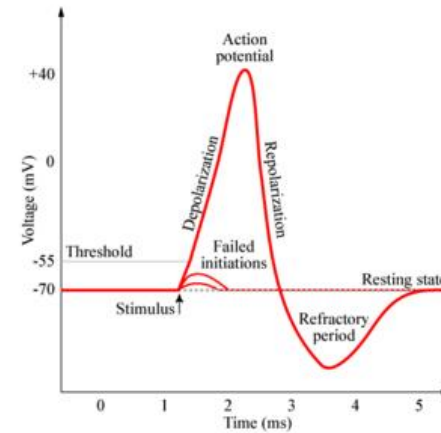
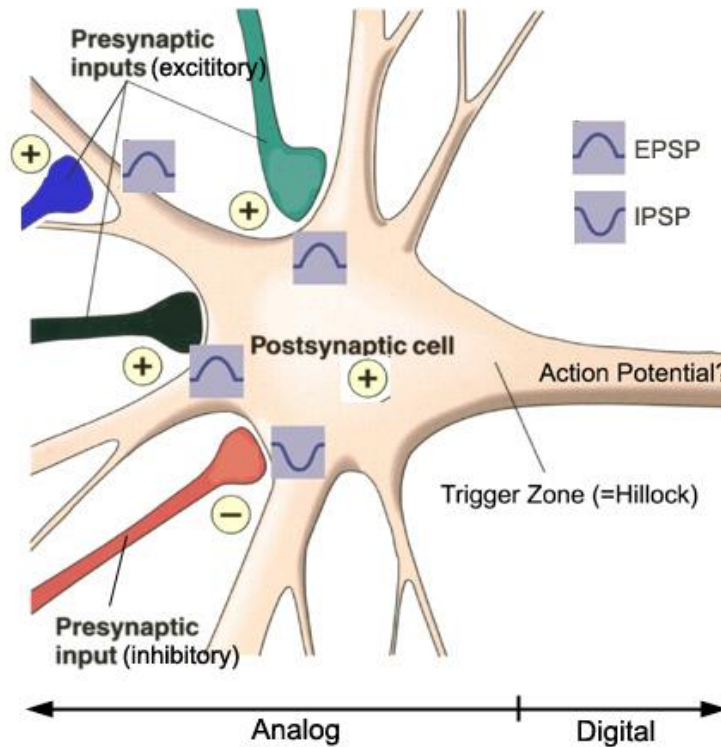
Input layer is not counted (does not contain any actual computational units) so this is a two-layer net:



Like the Associative (Willshaw) net, perceptrons calculate a weighted sum of their inputs.

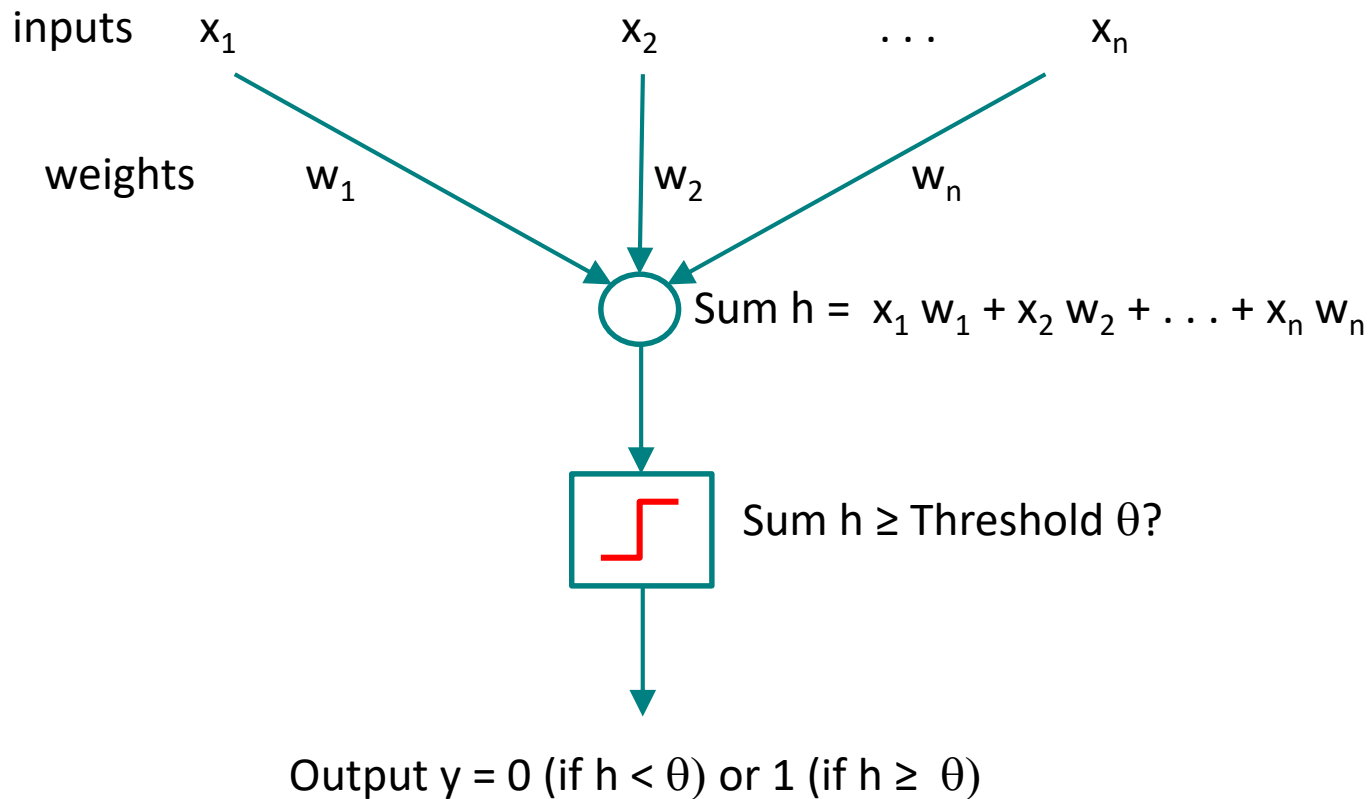
Summation of inputs in real and artificial neurons

Neurons fire an action potential if the sum of their inputs exceeds a threshold.



Summation of inputs in real and artificial neurons

All-or-nothing character of the action potential is reflected by threshold binary units (also called threshold boolean units or threshold logic units).



Threshold binary unit example

Inputs $x_1 = 0.5$
 $x_2 = 0$
 $x_3 = 1$

Weights $w_1 = 1$
 $w_2 = 2$
 $w_3 = 0.25$

Threshold $\theta = 0.5$

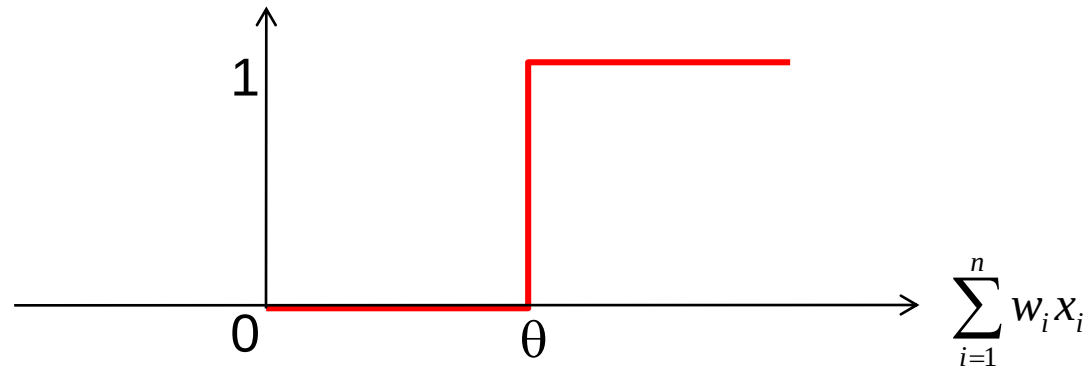
Sum $h = 0.5 \times 1 + 0 \times 2 + 1 \times 0.25 = 0.75$

Output $h \geq \theta$, therefore $y = 1$

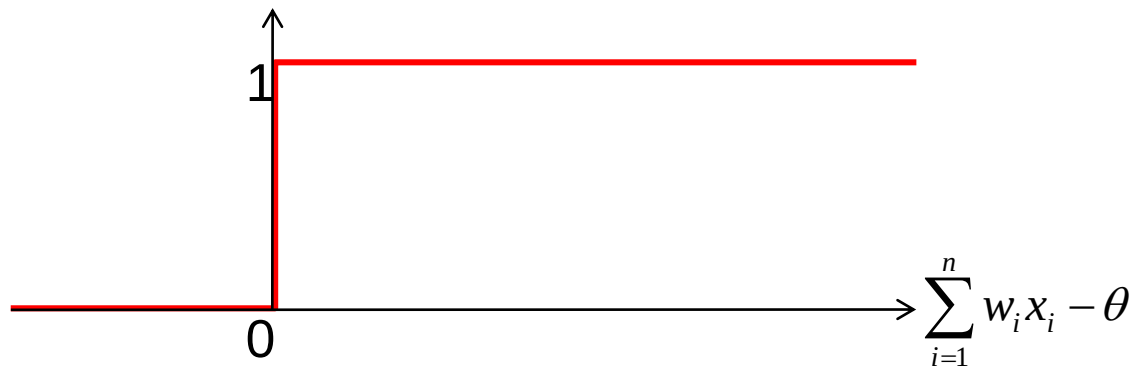
Shorthand notation

$$y = 0 \text{ if } \sum w_i x_i < \theta$$

$$y = 1 \text{ if } \sum w_i x_i \geq \theta$$



$$y = H\left(\sum_{i=1}^n w_i x_i - \theta\right)$$



Heaviside function: $H(x) = 0$ if $x < 0$, $H(x) = 1$ if $x \geq 0$

What can perceptrons compute?

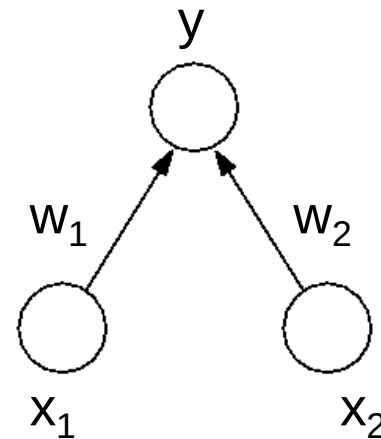
Perceptrons are feed-forward networks with separate input and output units: they can perform hetero-association tasks.

A typical application is pattern classification: for one output, only switch it on if the input pattern belongs to a certain class. For several output units, switch on different outputs for different classes of inputs.

Example: simple logical operations

x_1	x_2	$y = x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1

x_1	x_2	$y = x_1 \text{ OR } x_2$
0	0	0
0	1	1
1	0	1
1	1	1



Noise resistance

Resistance to noise is one of the most prominent features of neural networks and real brains.

Example: Threshold Binary Unit with $w_1 = 0$, $w_2 = 1$, $\theta = 0.5$

What is the response to input patterns $(x_1, x_2) = (0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$?

What happens if the weights are degraded to $(w_1, w_2) = (0.3, 0.7)$?

What happens with noisy inputs $(x_1, x_2) = (0.3, 0.3)$, $(0.3, 0.7)$, $(0.7, 0.7)$, $(0.7, 0.7)$?

What if both weights and inputs are noisy?

Noise resistance

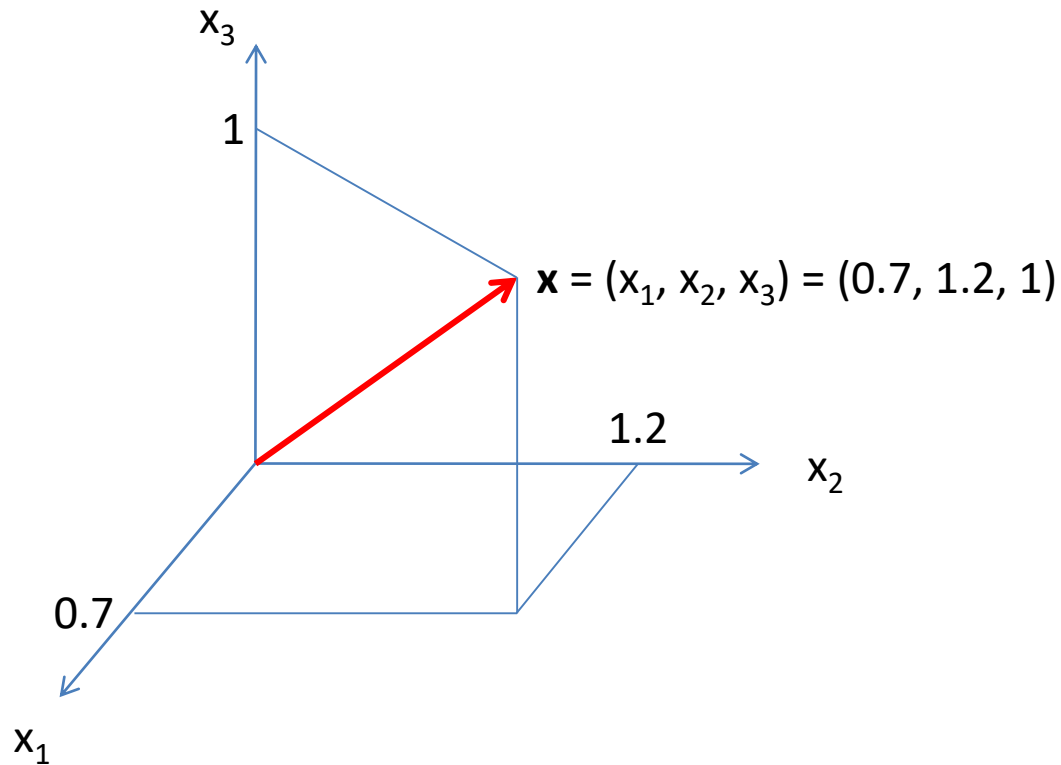
Example: Threshold Binary Unit with $w_1 = 0$, $w_2 = 1$, $\theta = 0.5$

w_1	w_2	x_1	x_2	y
0	1	0	0	
0	1	1	0	
0	1	0	1	
0	1	1	1	
0	1	0.3	0.3	
0	1	0.7	0.3	
0	1	0.3	0.7	
0	1	0.7	0.7	
0.3	0.7	0.3	0.3	
0.3	0.7	0.7	0.3	
0.3	0.7	0.3	0.7	
0.3	0.7	0.7	0.7	

A geometrical picture

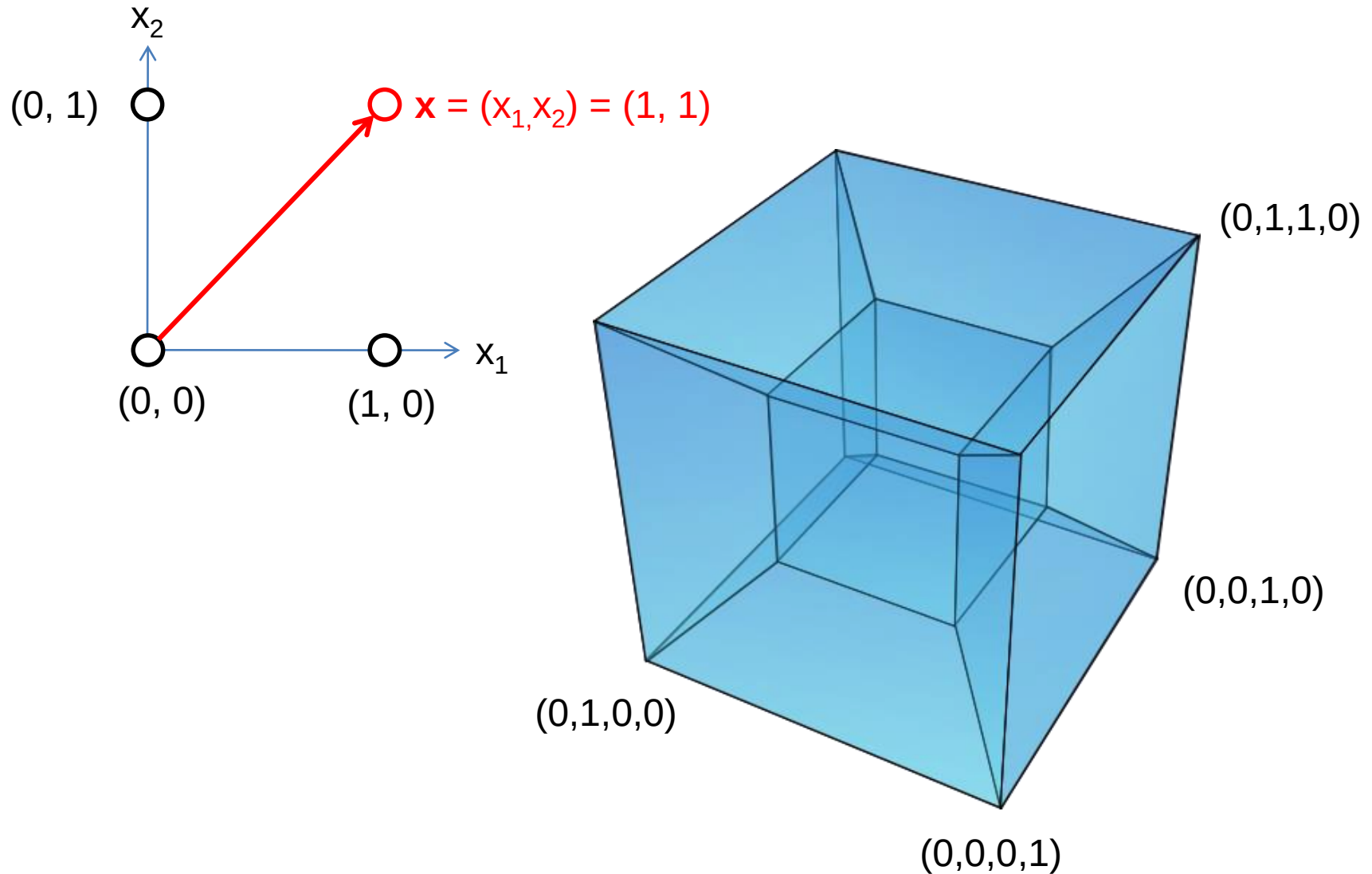
Each input unit can be considered as defining an axis. N input units define an N-dimensional input space.

A pattern of activity across the N input units can be represented as N-dimensional vector to a single point in this input space.



Input space

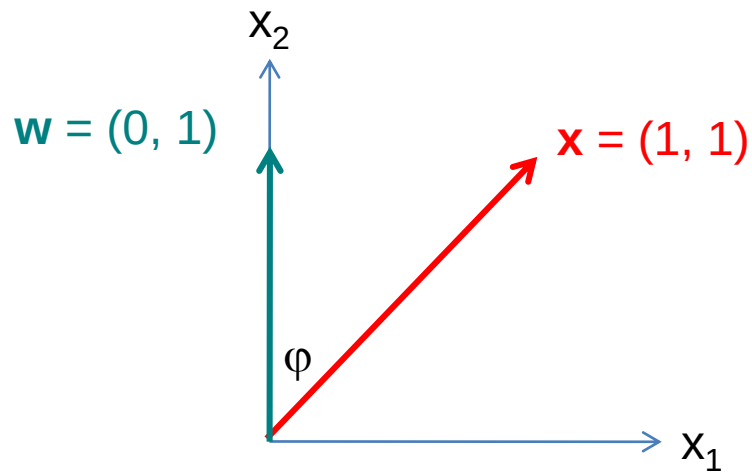
For binary inputs, each input pattern is represented by the corner of a hypercube.



Vectors and inner products

The weighted summation of inputs can also be represented using the inner product (or dot product) of the input vector and the weight vector: $\sum w_i x_i = \mathbf{w} \mathbf{x}$

We can plot the weight vector in input space:

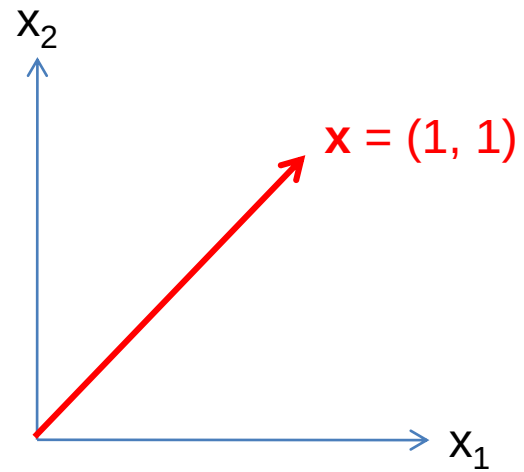


$$\mathbf{w} \mathbf{x} = 0 \times 1 + 1 \times 1 = 1$$

The inner product measures the similarity between the two vectors (why?).

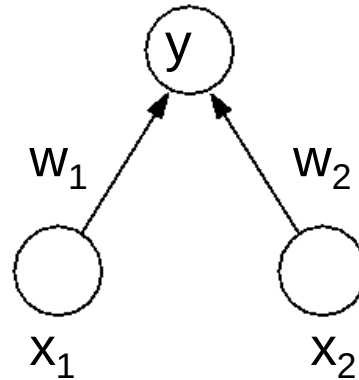
$$\mathbf{w} \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \varphi$$

Vectors and inner products



The weights and threshold define a decision surface

For a threshold binary unit with two inputs: $y = 1$ for $x_1 w_1 + x_2 w_2 \geq \theta$



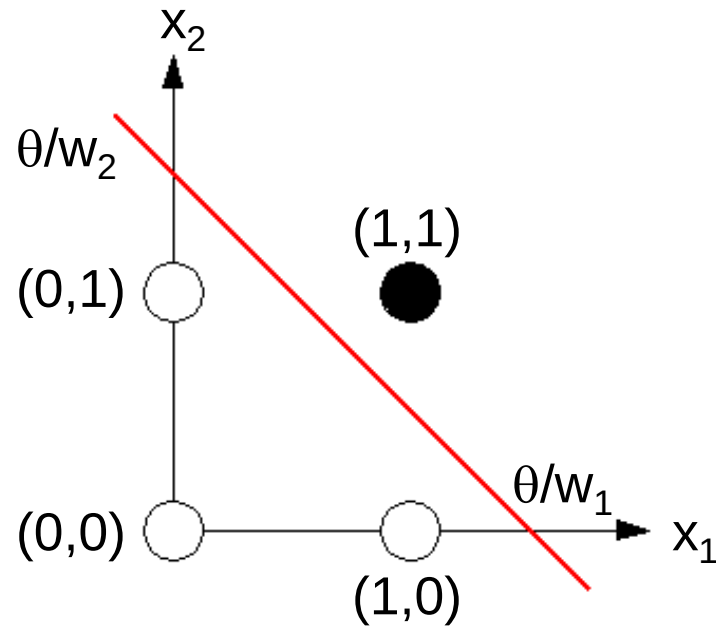
Thus, the output changes when $x_1 w_1 + x_2 w_2 = \theta$

Rearranging this gives $x_2 = -w_1/w_2 x_1 + \theta/w_2$

This is the equation for a line ($y = m x + b$) with slope $-w_1/w_2$ and intercept θ/w_2

Decision surfaces

For a perceptron with $w_1 = w_2 = 0.5$, $\theta = 0.75$



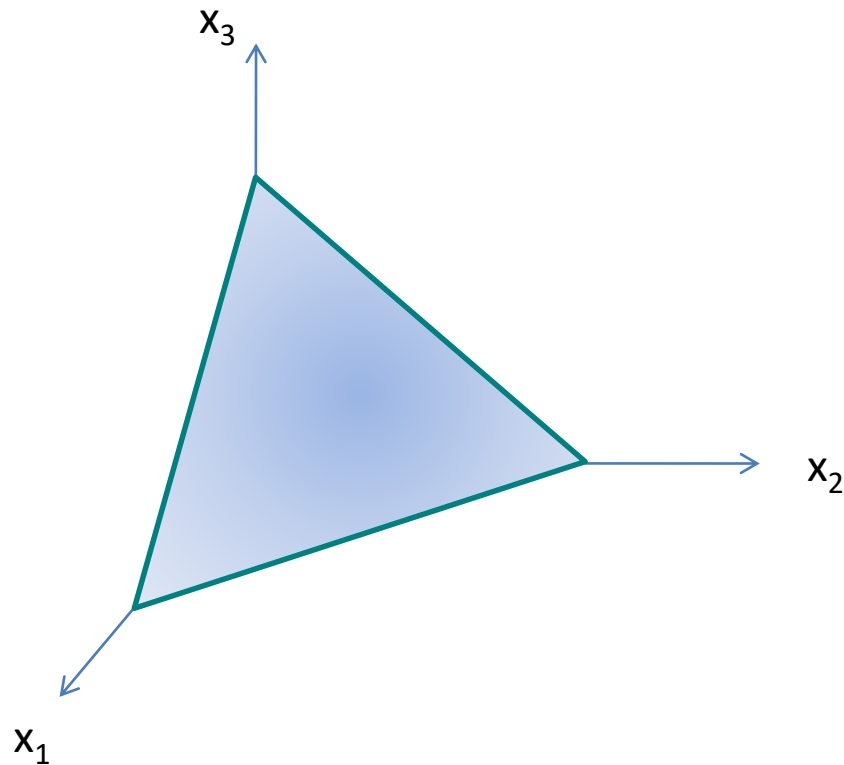
What does this compute?

Decision surfaces in higher dimensions

For three inputs: decision surface defined by $x_1 w_1 + x_2 w_2 + x_3 w_3 = \theta$

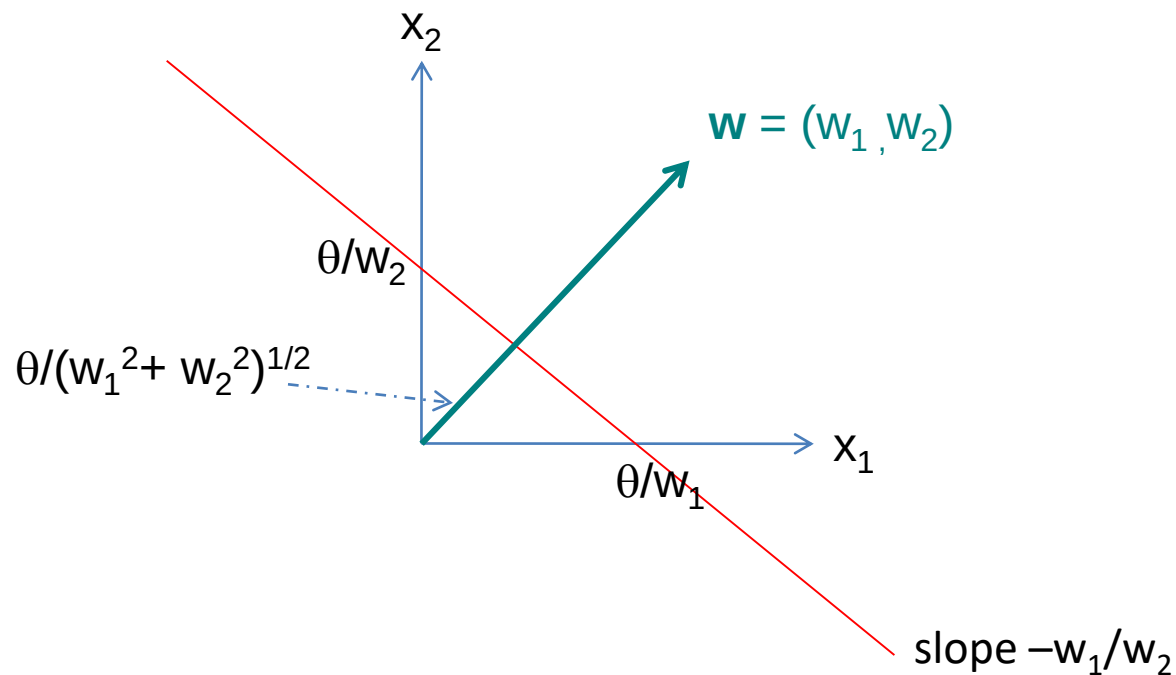
This is the equation for a plane (what are the intercepts?).

For N inputs: the decision surface is a hyperplane in N-dimensional space.



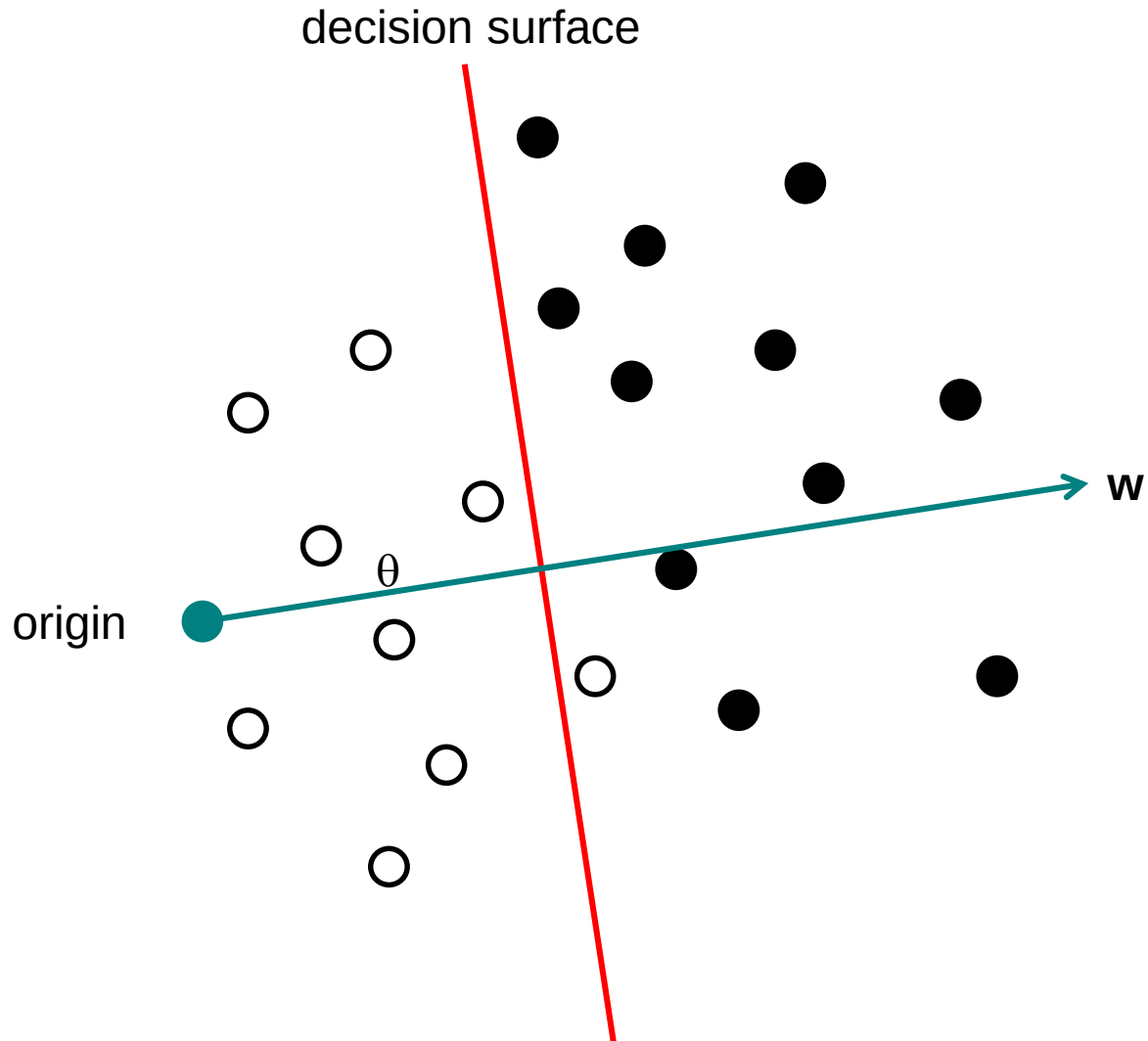
Decision surfaces and weight vectors

What is the relation between the weight vector and the location of the decision surface? Plot the weight vector in input space.



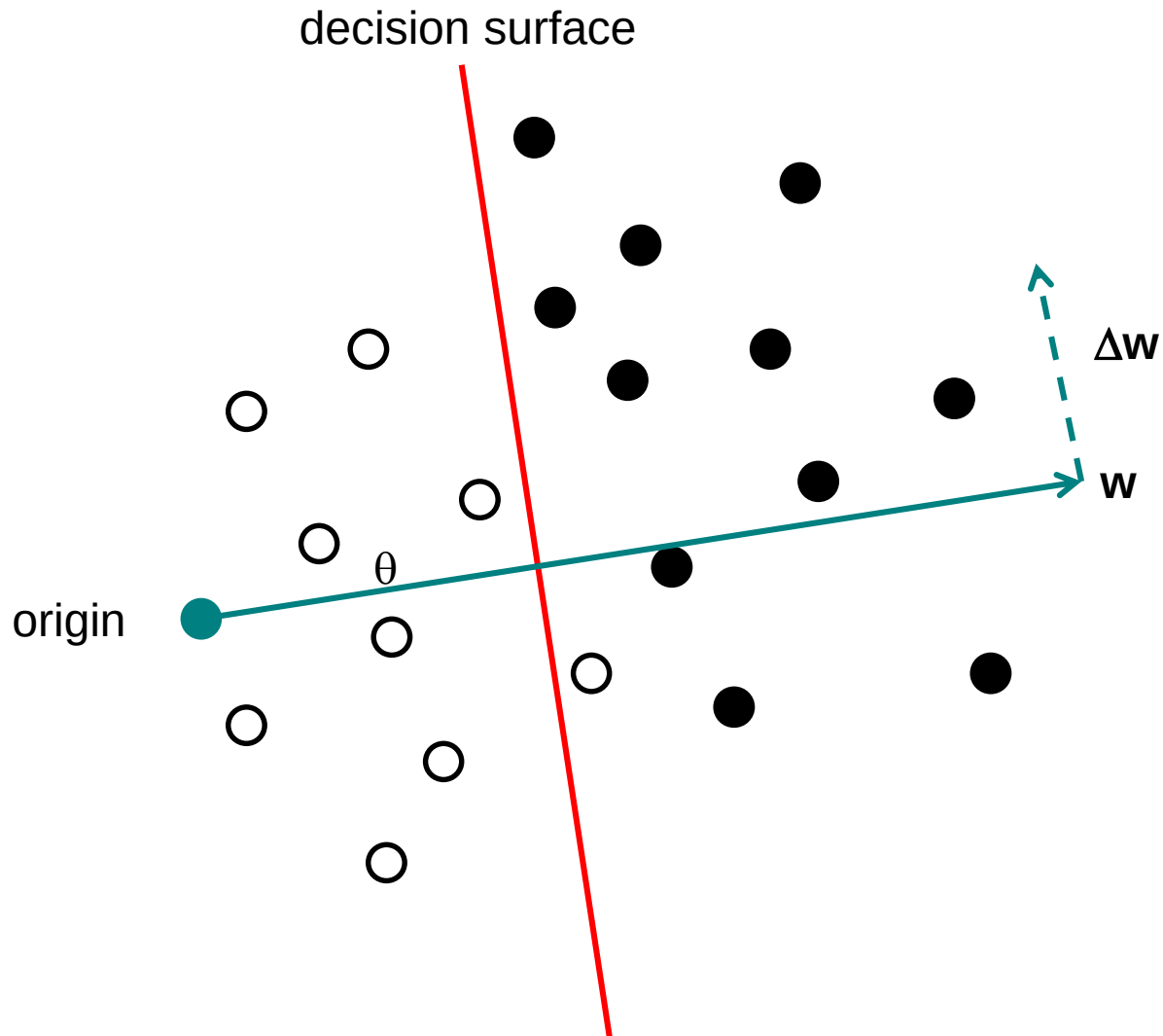
The decision surface is perpendicular to the weight vector and displaced from the origin in proportion to θ .

N-dimensional input space



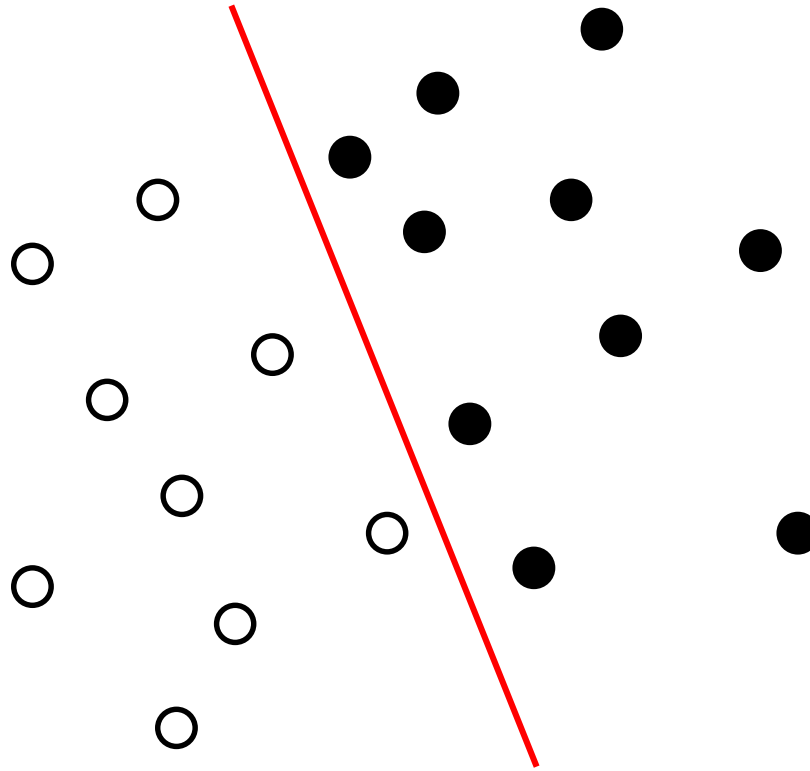
- Inputs for which the output should be 1
- Inputs for which the output should be 0

Learning by jiggling the weight vector



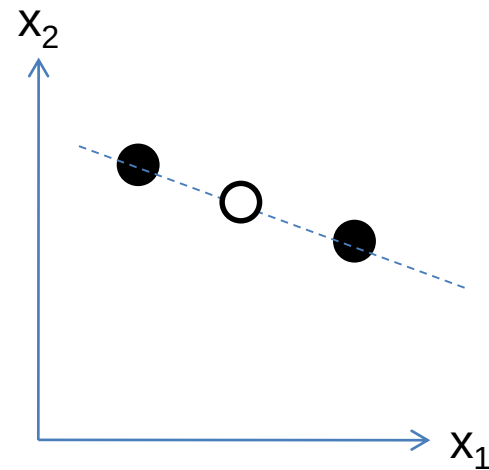
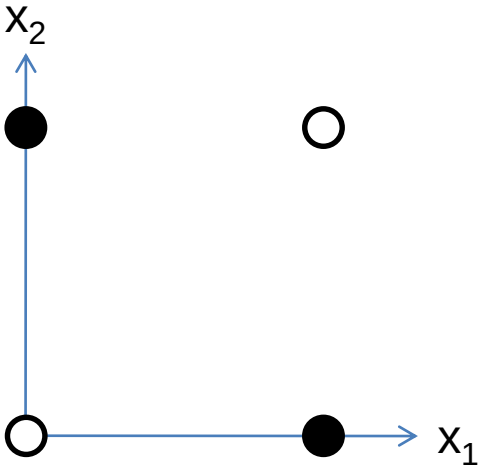
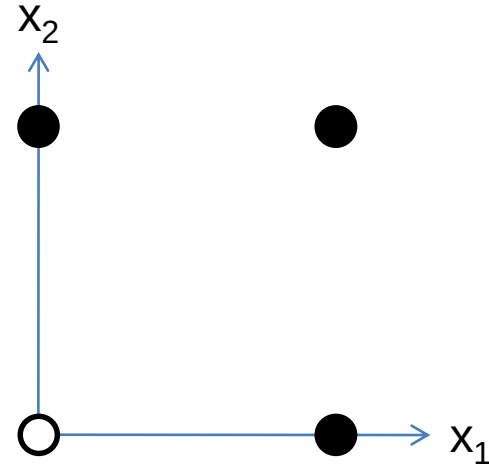
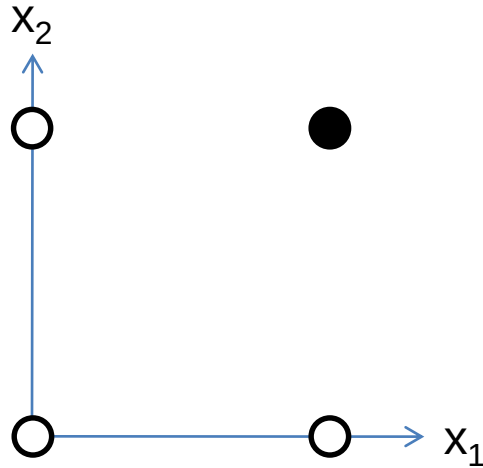
Linear separability

Binary classification can only be learnt if there is a decision surface that can separate inputs with targets 1 and 0.



Such a pattern classification is called linearly separable.

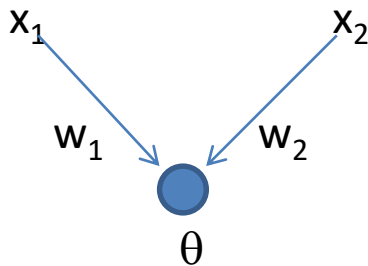
Examples of linearly separable and not linearly separable classifications



Threshold and bias: a simplification

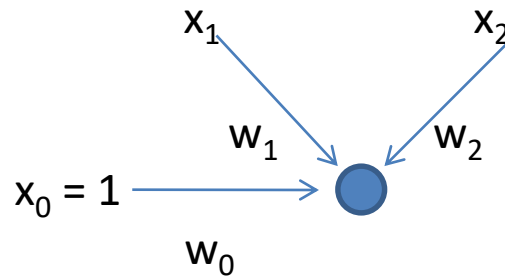
Having a weight vector and a separate threshold can be inconvenient.

The threshold can be replaced by a weight (bias weight w_0) from a new bias unit x_0 which is always ON.



$$\sum_{i=1}^N w_i x_i \geq \theta$$

$$\sum_{i=1}^N w_i x_i - \theta \geq 0$$



$$\sum_{i=0}^N w_i x_i \geq 0$$

The bias weight w_0 then becomes the negative of the threshold.

How can we learn the correct weights?

Try to develop a learning rule using common sense.

Want to learn a set of hetero-associations between a set of input patterns $\mathbf{x}^p = (x^p_1, x^p_2, \dots, x^p_N)$ and their target outputs t^p . How do we modify the weight vector $\mathbf{w} = (w_1, w_2, \dots, w_N)$?

Algorithm:

```
repeat
  for each association  $\mathbf{x}_p, t_p$ 
    calculate actual output  $y_p = f(\mathbf{x}_p, \mathbf{w})$ 
    if ( $y_p \neq t_p$ )
      update  $\mathbf{w} = \mathbf{w} + \Delta\mathbf{w}$ 
    end
  end
until ( $y_p = t_p$ ) for all p
```

How to calculate $\Delta\mathbf{w}$?

The Perceptron Learning Rule (Rosenblatt 1962)

How to determine Δw_i ? Important to consider the contribution of w_i to $\sum w_i x_i$.

If $x_i^p = 0$: do nothing, there is no contribution to $\sum w_i x_i$

If $y^p = t^p$: do nothing (the classification is correct)

If $y^p = 1$ and $t^p = 0$: $\sum w_i x_i$ is too large. Decrease all w_i associated with $x_i = 1$

If $y^p = 0$ and $t^p = 1$: $\sum w_i x_i$ is too small. Increase all w_i associated with $x_i = 1$

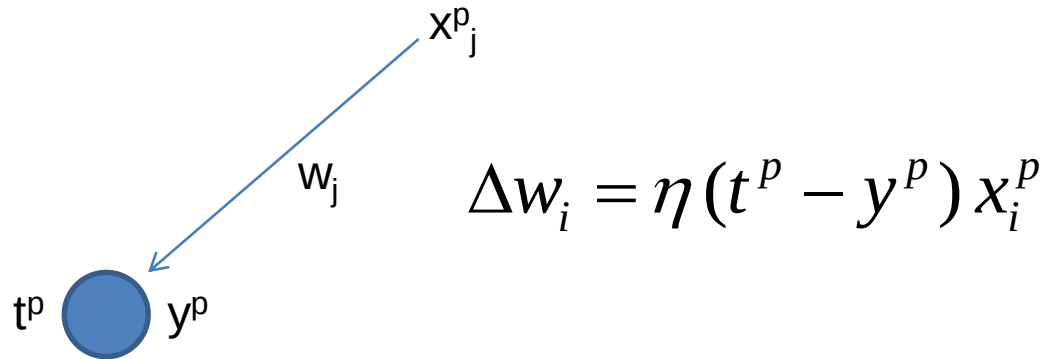
This can be summarised by:

$$\Delta w_i = \eta (t^p - y^p) x_i^p$$

where η is the learning rate.

Features of the Perceptron Learning Rule

A local rule: uses information that is locally available to the weight.



An error correction rule, and stable if no errors are made.

Guaranteed to find the correct weights. Perceptron Convergence Theorem: if a set of weights exist that separate the patterns then the rule will find them in finite time.

Is the correct set of weights unique? If not, how many sets of weights exist that can solve a given pattern classification?

Demo: initialise simulator with $w = (-0.3, 0.0, 0.4)$. Try to find correct weights that calculate AND.

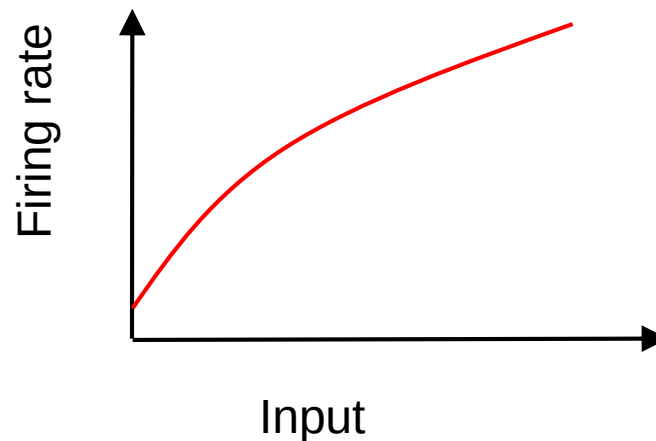
Other types of units

Until now we have restricted ourselves to threshold binary units.

One of the motivations for this was that real neurons have a threshold for action potential generation, and action potentials are all-or-none events. Thus, the output of a threshold binary unit represents the occurrence of an action potential (or not).

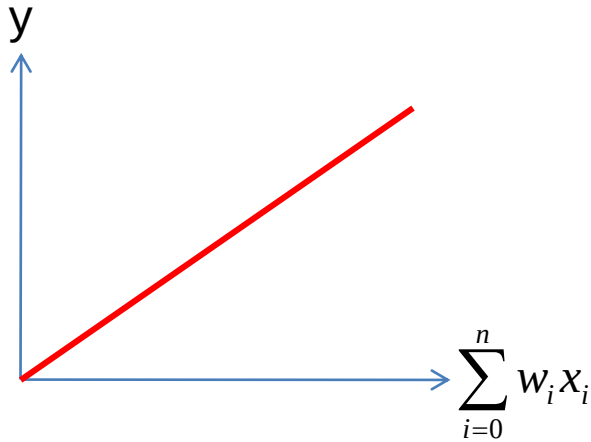
What if the output of the perceptron represents something else, for example the firing rate of the neuron?

Remember rate coding:



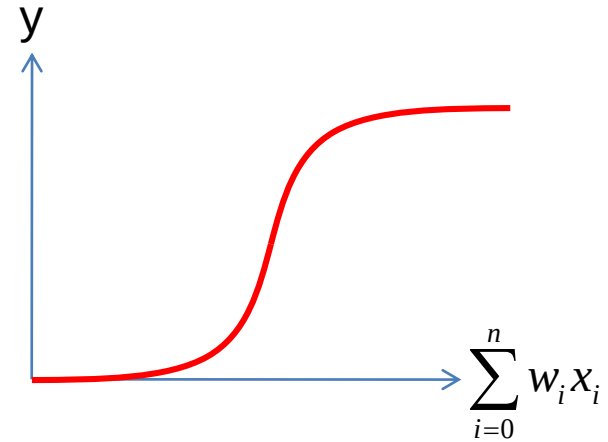
Linear units and sigmoid units

Linear activation function



$$y = \sum_{i=0}^n w_i x_i$$

Sigmoid activation function



$$y = \frac{1}{1 + \exp(-\sum_{i=0}^n w_i x_i)}$$

Another method for determining a learning rule

Can use an error function or cost function to measure the performance of the perceptron.

Error function must reflect how well the systems performs for a given set of weights.

Training is performed by adjusting the weights so as to minimise the value of the error function E .

How should such an error function look like?

Error function

Error function needs to be calculated over all input patterns p .

Should be 0 if $t^p = y^p$ for all p .

Define error function
$$E = \frac{1}{2} \sum_p (t^p - y^p)^2$$

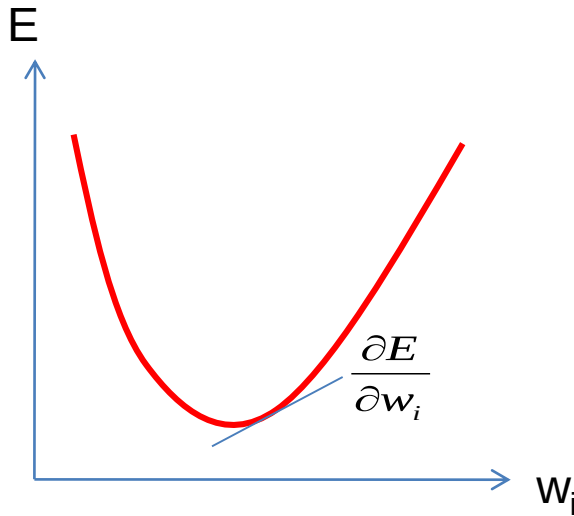
Now need to determine how E varies with the weights $\mathbf{w} = (w_1, w_2, \dots, w_N)$

Calculate the change of E when weight w_i is changed

This is measured by the (partial) derivative $\frac{\partial E}{\partial w_i}$

Weight update using an error function

How to change w_i to minimise E ?



$$E = \frac{1}{2} \sum_p (t^p - y^p)^2$$

When w_i is changed by an amount Δw_i the resulting change in E is $\Delta E = \frac{\partial E}{\partial w_i} \Delta w_i$

If we make Δw_i proportional to $-\frac{\partial E}{\partial w_i}$

then $\Delta E = -\left(\frac{\partial E}{\partial w_i}\right)^2$ i.e. E always decreases

Calculating the weight update for linear units

First need to calculate $\frac{\partial E}{\partial w_i}$

$$E = \frac{1}{2} \sum_p (t^p - y^p)^2$$

$$\frac{\partial E}{\partial w_i} = - \sum_p (t^p - y^p) \frac{\partial y^p}{\partial w_i}$$

$$\frac{\partial y^p}{\partial w_i} = \frac{\partial (\sum_j w_j x_j)}{\partial w_i} = x_i$$

Therefore $\frac{\partial E}{\partial w_i} = - \sum_p (t^p - y^p) x_i^p$

and $\Delta w_i = -\eta \frac{\partial E}{\partial w_i} = \eta \sum_p (t^p - y^p) x_i^p$ (for all patterns)

For a single pattern p: $\Delta w_i = \eta (t^p - y^p) x_i^p$

This is the Delta Rule or Widrow-Hoff Rule and in fact the same as the Perceptron Learning Rule for threshold binary units that was derived using common sense.

Limitations of single layer networks

Many pattern classification problems are not linearly separable.

There are 2^N possible binary input patterns.

For threshold binary units this results in 2^{2^N} possible classifications.

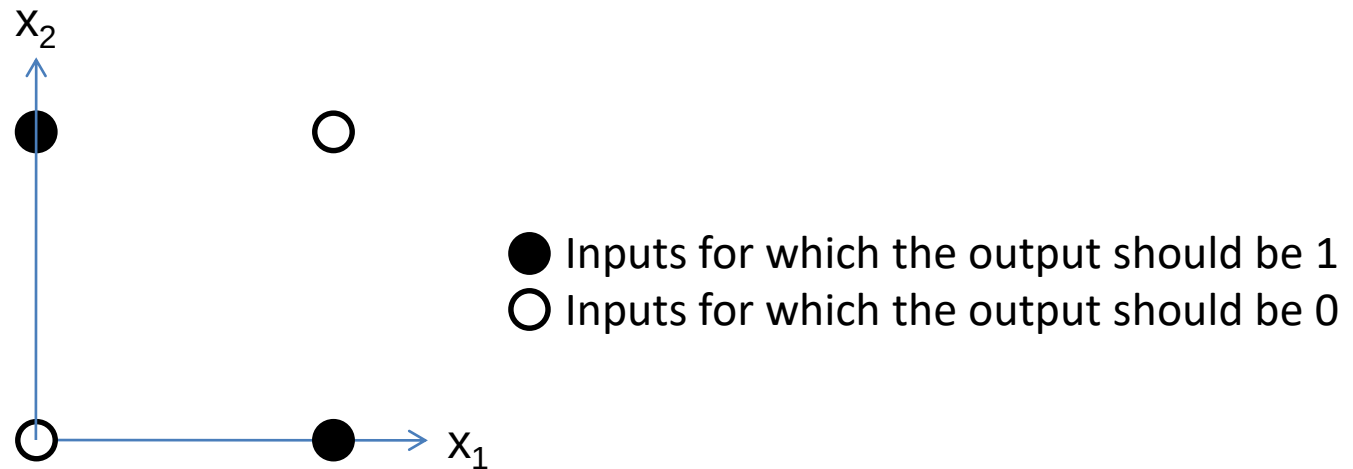
Only a fraction of these are linearly separable.

With high-dimensional input patterns there is no easy way to tell whether a given problem is linearly separable.

Example of a problem that is not linearly separable?

The XOR problem

x_1	x_2	$y = x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0

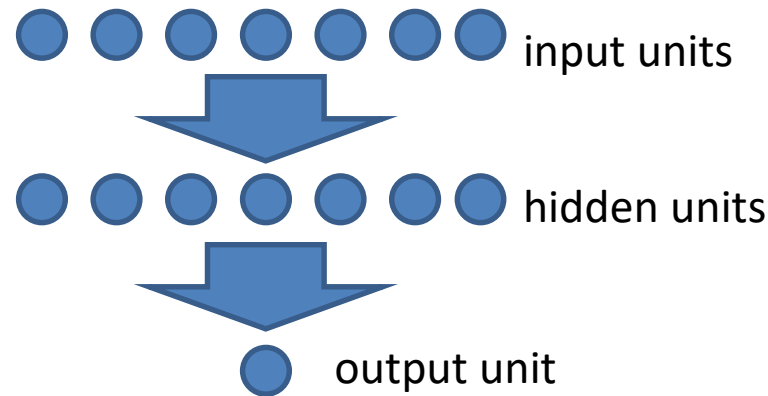


No decision surface exists that separates input patterns with $t = 1$ from input patterns with $t = 0$. How can we solve this problem?

The power of multi-layer networks

Consider binary classifications of binary input patterns.

Consider a network with a single output unit and a large number of hidden units.



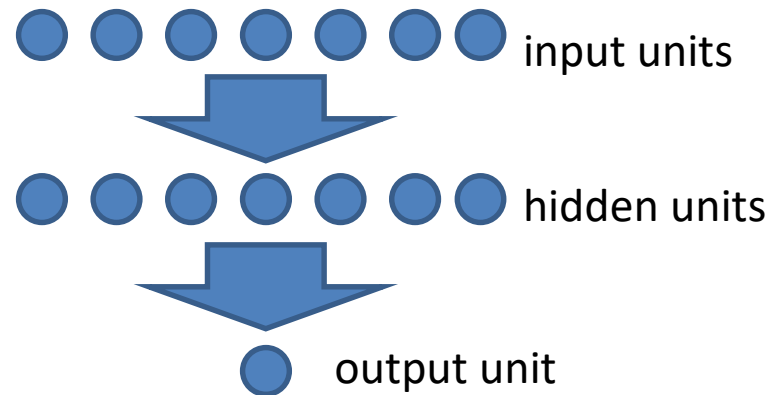
We can wire up the hidden units so that each unit responds to just one input pattern which has a target = 1.

Then have the output unit calculate the logical OR of its inputs.

In principle, any binary classification is computable by a network with hidden units.

In practice, such a look-up table approach has its disadvantages (what are they?).
How can we train networks with hidden units?

Finding weights in multi-layer networks



Perceptron learning rule requires knowledge of the target output for each input pattern.

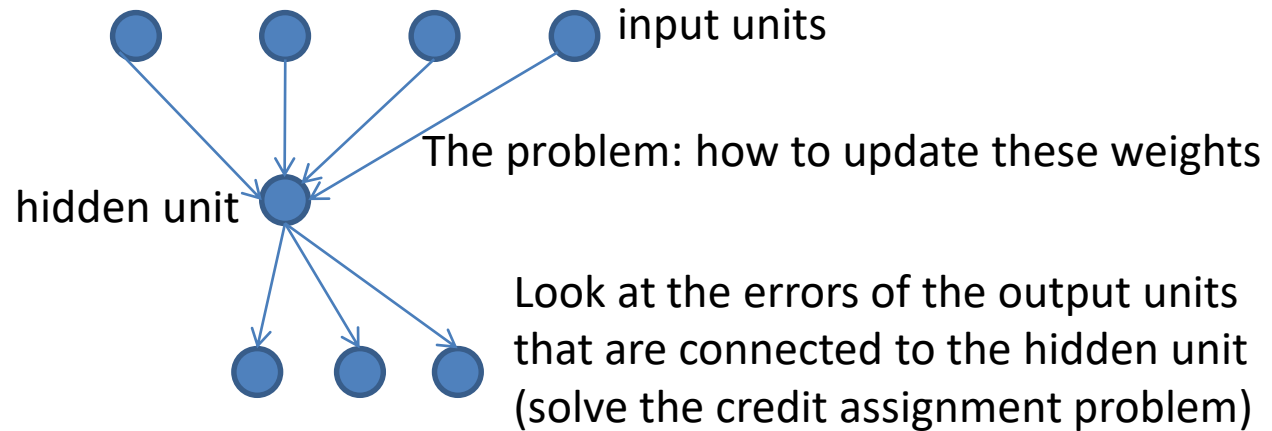
This is no problem for the output unit: can update the weights between hidden units and output unit as before (for weights between input units and output unit in the single layer network).

How about the weights between input units and hidden units? What are the targets for the hidden units?

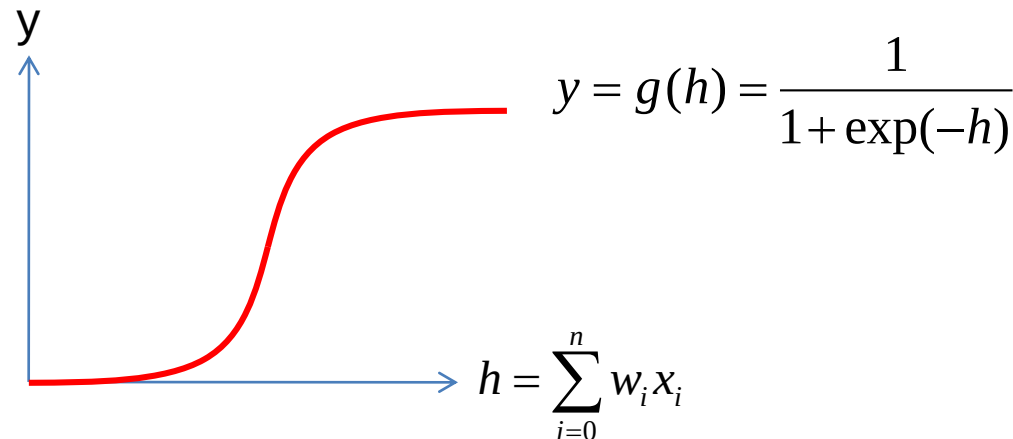
Problem is solved by Back Propagation Learning Algorithm.

Back Propagation Learning Algorithm

Rumelhart, Hinton, Williams (1985)



Use units with non-linear activation functions (otherwise no benefit from hidden units – why?).



Weight updates using error back propagation

As before, use error measure $E = \frac{1}{2} \sum_p (t^p - y^p)^2$

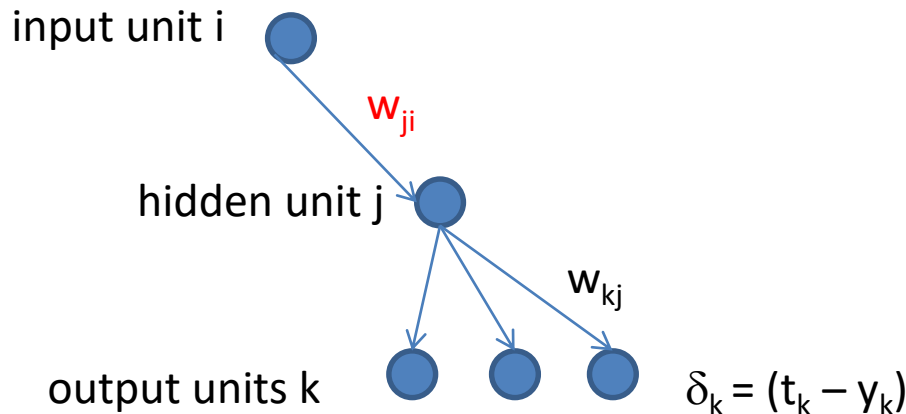
and start with $\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$ (change weights so as to decrease error)

Calculate derivatives using chain rules as before. This gives us (for all weights)

$$\Delta w_{ji} = \eta \delta_j^p x_i^p$$

For weights between hidden units and output units $\delta_j^p = (t_j^p - y_j^p) g'(h_j^p)$

For weights between input units and hidden units $\delta_j^p = \sum_k w_{kj} \delta_k^p g'(h_j^p)$

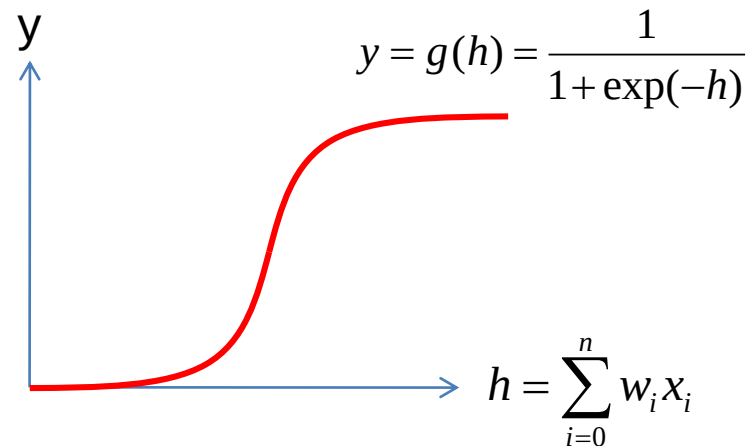


Back Propagation Problems

Back propagation of the error signal is biologically implausible?

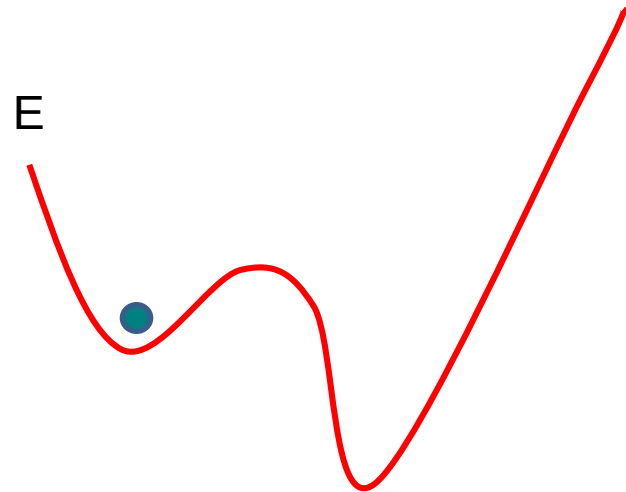
Can become trapped in local minima of the error function.

Often takes too long to find the correct weights.



$$\Delta w_{ji} = \eta \delta_j^p x_i^p$$

$$\delta_j^p = (t_j^p - y_j^p) g'(h_j^p)$$



Back Propagation Modifications

Lower the learning rate with time (bypass local minima at first, then settle into global minimum).

Add a momentum term to the weight update (speeds up progress along shallow gradients).

$$\Delta w_{ji}(t) = -\eta \frac{\partial E}{\partial w_{ji}} + \alpha \Delta w_{ji}(t-1)$$

Knock the system out of local minimum by adding noise (allow occasional upward steps of E or randomly perturb $\mathbf{w}$).

Use other error functions or activation functions.

Exercise 1: simple perceptron

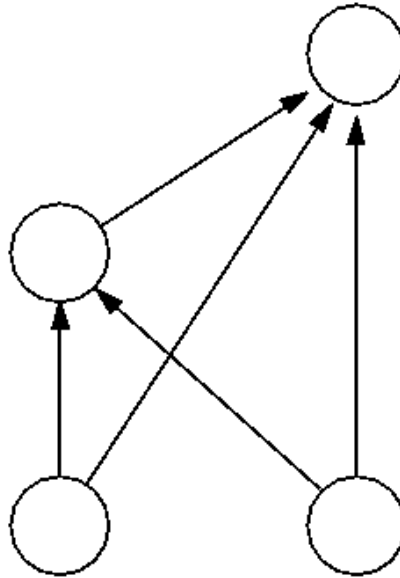
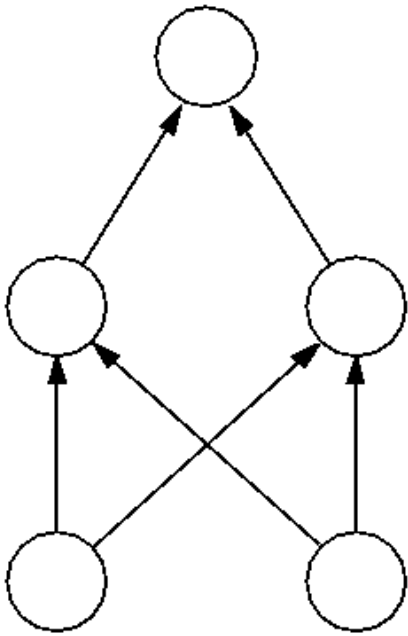
(1) Draw the input space for the following classification:

x_1	x_2	target
0	0	0
0.5	0	0
0	1	1
1.5	0	1

(2) Draw a possible decision line in input space. What is the equation describing this line, and what are possible weights and a threshold that would result in such a decision line?

Exercise 2: Using a two-layer network to solve XOR

- (1) Find possible weights and thresholds that solve the XOR problem.
- (2) Draw the decision lines in input space.



x_1	x_2	$y = x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0